

V.32 Atomic Interop Audit



September 9, 2026

This document has been prepared solely for the named client and it may not be relied upon by any third parties. The content in this document is provided for informational purposes only and does not constitute professional advice of any kind. Security assessments are limited by factors such as scope, time, and techniques used and therefore this document does not constitute a certification, guarantee, or warranty regarding the proper functioning of any system or the absence of security vulnerabilities.

Copyright © 2026 Zeppelin Group Ltd.

Table of Contents

Table of Contents	2
Summary	3
Scope	4
System Overview	5
Atomic Interop (PR#2311, PR#2315)	5
In-Place Upgrade Parity (PR#2344)	6
L2 Asset Bookkeeping Redesign (PR#2364)	6
Interop-Only Bridging and Verifier Cleanup (PR#2220)	7
Security Model and Trust Assumptions	8
Low Severity	10
L-01 Atomic Sends Can Be Griefed By Invalidation Of Low-Nullifier Hint	10
L-02 Funds Burned By Non-Router Indirect Call Starters Cannot Be Recovered On Timeout	10
Notes & Additional Information	11
N-01 Default refundRecipient Resolution Is Inconsistent With Sender Aliasing	11
N-02 Out-of-Range _missingLegIndex Reverts With a Generic Panic	12
N-03 parseAtomicSend Iterates Over All Attributes Even After Finding a Match	12
N-04 Redundant Re-Parsing Of Settlement Proof Per Atomic Interop Leg	13
N-05 Incorrect Argument Passed To CannotInitiateInteropOnL1	13
N-06 Redundant And Dead Code In Multiple Contracts	14
N-07 Redundant Read of the Legacy Shared Bridge Address	14
Conclusion	16
Appendix	17
Issue Classification	17

Summary

Type	Layer 2 & Rollups	Total Issues	9 (9 resolved)
Timeline	From 2026-08-06 To 2026-08-18	Critical Severity Issues	0 (0 resolved)
Languages	Solidity	High Severity Issues	0 (0 resolved)
		Medium Severity Issues	0 (0 resolved)
		Low Severity Issues	2 (2 resolved)
		Notes & Additional Information	7 (7 resolved)
		Client Reported Issues	0 (0 resolved)

Scope

OpenZeppelin performed a diff audit of the [matter-labs/era-contracts](#) repository. In scope were the production solidity files of the following pull requests:

- [pr#2311](#) at commit [048cb05](#)
- [pr#2315](#) at commit [1709625](#)
- [pr#2344](#) at commit [44be7d9](#)
- [pr#2364](#) at commit [50f761d](#)
- [pr#2220](#) at commit [86e18f8](#)

Update: We verified that commit [961675f](#) of the [release/v0.33.0-atomic-interop](#) branch contains all the fixes for the issues reported during this assignment.

System Overview

The changes under review are part of the v32 release. The release builds on the interop foundation introduced in v31, in which cross-chain messages are sent as bundles through the [InteropCenter](#) on the source chain and verified against an aggregated message root on the destination.

The release makes the interop bundle path the only path for cross-chain transfers and removes the legacy bridging paths entirely, retaining deprecated storage placeholders only where existing storage layouts require them. Alongside this, the L2 asset bookkeeping was redesigned and the in-place upgrade from v31 was made to produce the same end state as a fresh v32 deployment. The main changes under review were as follows.

Atomic Interop ([PR#2311](#), [PR#2315](#))

Atomic interop makes a cross-chain flow that spans several chains all-or-nothing. A flow is made up of legs, one per source chain, and the calls of any leg may execute on their destination only if every leg of the flow was committed on its own source chain before a shared deadline. If that does not happen, the legs that did commit become refundable. Atomic bundles are now the only form of L2 to L2 interop; the sole remaining non-atomic send is an L2 to L1 withdrawal.

This is achieved without a coordinating contract on L1. Each chain instead keeps an append-only indexed Merkle tree, the [L2InteropCommitmentTree](#), into which the [AtomicFlowManager](#) records one leaf per leg. The tree's root is snapshotted by the bootloader at every batch boundary, settles into the settlement layer's message root, and is imported back into every chain over the interop root channel that already existed. Finality is therefore proven by the destination rather than announced to it, and no new L1 contract or global registry is introduced.

A flow is identified by a [flowId](#) derived from all of its legs' bundle hashes, their source chains, the deadline, and the settlement layer. The sender supplies this data as a bundle attribute travelling alongside the bundle rather than inside it, which lets the flow manager recompute the identifier on-chain and confirm that the bundle being sent is genuinely one of the flow's legs. Sending a leg inserts its commitment leaf into the source chain's tree and marks the leg as committed.

On the destination, execution requires a proof that every leg was committed in its own source chain's tree as of a batch that settled before the deadline. If a leg never commits, one proof of that leg's absence past the deadline authorizes the unwind of the whole flow, since a flow that cannot finalize anywhere cannot finalize at all. Each chain then marks its own committed legs as revertable, and a follow-up claim asks the original sender of each call to reverse its own burn. This recovery is best-effort by design: the asset router implements the reversal hook for the transfers it created, while native value sent by direct calls is refunded through a separate path. In this release every participating chain must settle on L1, so the deadline and both timestamps used by the proofs are L1 block times.

In-Place Upgrade Parity ([PR#2344](#))

This change closes the gaps that would leave an upgraded ecosystem in a different state than one deployed from scratch. Address discovery in `AddressIntrospector` became version-aware, reading `chainRegistrationSender` from v31 onward but `l1InteropHandler` only from v32 onward, so that the upgrade can be prepared against a live v31 ecosystem. The upgrade now deploys `L1InteropHandler` when the ecosystem does not have one, wires it into both `L1Nullifier` and `L1AssetRouter`, and accepts its ownership. It also sets the Bridgehub's registration sender when none is configured, and preserves an existing `ChainRegistrationSender` proxy by refreshing only its implementation.

The most significant addition is `L1NativeTokenVault.populateBridgedOut`, which folds pre-upgrade accounting into the `bridgedOut` mapping once per asset. Without it, the mapping starts empty on an upgraded vault and every withdrawal of an L1-native asset reverts. The amounts are derived on-chain, either from the complement of L1's own balance bucket in the retired asset tracker or from the vault's own deprecated per-chain balances. Finally, the atomic interop built-ins are force-deployed and initialized for ZKsync OS chains on the upgrade path. Era chains deliberately do not receive atomic interop, as their bootloader does not support it.

L2 Asset Bookkeeping Redesign ([PR#2364](#))

This change reworks how chain-local asset accounting is stored, and is an alternative to an earlier proposal that had moved this state into the contracts assets already flow through. Here, `L2AssetTracker` is kept as a separate system contract that hosts the bookkeeping, so that `L2NativeTokenVault` remains focused on escrow integrity.

`L2AssetTracker` holds chain-local, write-mostly bookkeeping that no bridging decision reads: the correctness of transfers is guaranteed by ZK proofs rather than by these balances. It records a per-asset balance for tokens native to the chain, counters used to size the L1 to Gateway migration, and a snapshot of each token's total supply taken before its first post-v31 bridge operation. The native token vault and the base token holder report every inbound and outbound transfer to it, and it gates recovery of a failed bridge-out by asserting that the reversal leaves the accounting unchanged. The shared tracker base and the L1 and Gateway trackers were removed together with the cross-chain token balance migration they served, while unused inherited contracts and deprecated slots are kept so that the storage layout of deployed instances is preserved.

On L1 the equivalent accounting is a single mapping, `L1NativeTokenVault.bridgedOut`, holding the net amount of each L1-native token currently bridged out of L1. It rises when tokens leave L1 and falls when they return, so unlike a raw token balance it cannot be skewed by transfers sent directly to the vault. An inbound amount greater than the recorded one would imply that bridged representations were forged upstream, so such a transfer is rejected rather than recorded. The per-chain balances it replaces remain as deprecated state behind a backwards-compatible getter.

Interop-Only Bridging and Verifier Cleanup ([PR#2220](#))

This change makes the interop bundle path the only path for cross-chain transfers. All L2 to L1 withdrawals are routed through the `InteropCenter` on the source chain and finalized by `L1InteropHandler`, using the asset router's `finalizeDeposit` format for both tokens and base tokens. The standalone legacy withdrawal and finalization paths are removed together with their replay-protection logic, as are the legacy L1 ERC20 bridge, the L2 shared bridge compatibility layer, and the associated scripts, ABIs, and deployment parameters. Deprecated storage placeholders are retained where needed to preserve upgradeable storage layouts.

The release also simplifies proof verification on ZKsync OS chains. The version-indexed verifier management was replaced by a single immutable PLONK verifier, the unused FFLONK verifier was removed, and the dual verifier contract was renamed accordingly.

Security Model and Trust Assumptions

The system involves several privileged roles, typically designated as owner, admin, or upgrader, which are expected to differ across chains and contracts. These roles can set configuration parameters, pause parts of the protocol, and carry out upgrades and migrations. They were considered trusted entities within the scope of this review, and the correct functioning of the system depends on their proper, timely, and honest operation.

Below, we outline the main assumptions on which the changes under review rest.

- **Proof channel.** Both atomic interop and withdrawal finalization reduce to proving inclusion against interop roots imported from the settlement layer. These proofs are only as trustworthy as the imported roots and the batch execution that double-checks them on the settlement layer. Atomic interop adds no assumption beyond what that channel already required.
- **L1-only settlement for atomic flows.** `AtomicFlowManager` requires every flow to declare L1 as its settlement layer and re-checks this when a leg is appended, when finality is proven, and when a refund is authorized. The timeout argument depends on this, so extending atomic interop to chains settling on the Gateway would require revisiting it.
- **Chain-local balances are advisory.** Neither the `L2AssetTracker` balances nor `L1NativeTokenVault.bridgedOut` is what makes a transfer correct; that follows from the ZK proofs. `bridgedOut` still acts as a backstop on L1 by rejecting an inbound amount larger than the outstanding one, but no bridging decision consults the L2 tracker.
- **Upgrade execution.** An in-place upgrade is trusted to deploy and wire `L1InteropHandler` and to run the one-off `bridgedOut` population for every affected asset before withdrawals resume. `populateBridgedOut` is permissionless and skips assets already populated, but until it has run for an asset, withdrawals of that asset revert.
- **Newly configured addresses.** `setL1InteropHandler` on `L1Nullifier` and `L1AssetRouter` is restricted to the owner and can be set only once, so a wrong value cannot be silently replaced, but it also cannot be corrected without an upgrade. `Bridgehub.setAddressesV31` is callable by the owner or upgrader and can be re-pointed at any time.

- **Pausing.** `L1InteropHandler` is pausable by its owner. While paused, interop withdrawals cannot be finalized on L1.

Low Severity

L-01 Atomic Sends Can Be Griefed By Invalidation Of Low-Nullifier Hint

`IndexedMerkleTree.insert` walks the sorted linked list for at most `MAX_LOW_INDEX_SEARCH_ATTEMPTS` (5) hops before reverting with `IMTLowLeafNextTooSmall`. The hint it starts from is supplied by the sender in the `atomicBundle` attribute and forwarded unchanged through `dispatchBundle` and `AtomicFlowManager.append`, so it is computed off chain and may be stale by the time the transaction executes.

Since commit values are keccak digests, a third party can search for bundle salts that place six of their own values between a target's hint and its commit value, exhausting the search bound and reverting that send. The cost is six atomic sends, and the attack can be repeated against each retry. The same failure can also occur without any adversary, simply on a chain with frequent atomic sends.

Consider raising the search bound, or replacing the fixed hop count with a walk limited by remaining gas.

Update: Resolved in [pull request #2349](#).

L-02 Funds Burned By Non-Router Indirect Call Starters Cannot Be Recovered On Timeout

In `AtomicFlowManager`, the `_recoverBundle` function selects a recovery mechanism based on the call's local sender. It invokes `recoverAtomicCall` for indirect calls only when `from == L2_ASSET_ROUTER_ADDR`, and uses the base-token refund path only when `from != L2_ASSET_ROUTER_ADDR && value != 0`.

However, any contract implementing `IL2CrossChainSender` may serve as an indirect call starter, since `InteropCenter._processCallStarter` invokes `initiateIndirectCall` on the `user-supplied to` and sets `InteropCall.from` to that address. A leg whose funds were burned by any non-router starter therefore matches neither branch in `AtomicFlowManager._recoverBundle`: no reversal hook is called, and no value

is refunded. Since `claimRefund` marks the leg `Reverted` before dispatching recovery and ignores the outcome, the claim succeeds as a no-op and the burn becomes permanently unrecoverable.

The `IAAtomicRecoverable` interface and the requirements stated for its implementors in `protocol-docs/atomicity/recovery.md` ("the manager is agnostic to call/encoding formats ... implementations MUST gate both hooks to the canonical AtomicFlowManager") anticipate senders other than the asset router, but the address-pinned dispatch can never reach them.

Consider making recovery dispatch generic over `IAAtomicRecoverable` senders. Until that is feasible, consider documenting the restriction in `protocol-docs/atomicity/security.md` as an accepted limitation, and amending the sender-agnostic wording in `protocol-docs/atomicity/recovery.md` so it reflects that only the canonical asset router is currently recoverable.

Update: Resolved in [pull request #2348](#).

Notes & Additional Information

N-01 Default `refundRecipient` Resolution Is Inconsistent With Sender Aliasing

When `refundRecipient` is left unset, `BridgehubBase._sendRequest` substitutes `msg.sender` without applying the L1-to-L2 alias. `Mailbox` therefore always takes the non-zero branch within `actualRefundRecipientMailbox`, which aliases based on `code.length`, rather than the zero branch, which aliases based on `_originalCaller == tx.origin`.

The two checks disagree for a caller whose code is not yet deployed, such as a contract bridging from its own constructor. In that case the same address may be `aliased as request.sender` but not as `request.refundRecipient`. Callers that name a recipient explicitly are unaffected.

Consider aligning the default path on a single check so that an unset `refundRecipient` resolves to one L2 identity regardless of which contract performs the resolution, for consistency.

Update: Resolved in [pull request #2353](#).

N-02 Out-of-Range `_missingLegIndex` Reverts With a Generic Panic

In `AtomicFlowManager`, `authorizeRefund` indexes `preimage.legSourceChainIds[_missingLegIndex]` and `preimage.legBundleHashes[_missingLegIndex]` with a caller-supplied index value that is never bounds-checked. A value at or above the leg count causes a generic panic revert rather than throwing one of the contract's named errors, which is less informative for callers and off-chain tooling than the rest of the refund path.

Consider adding an explicit bounds check against `preimage.legBundleHashes.length` and reverting with a dedicated error in `AtomicInteropErrors.sol` for clarity and consistency.

Update: Resolved in [pull request #2350](#).

N-03 `parseAtomicSend` Iterates Over All Attributes Even After Finding a Match

In `InteropAttributeParser`, `parseAtomicSend` has no early exit: it continues scanning the remaining elements, reading and comparing each one's selector even after finding the `atomicBundle.selector` entry. Because the `atomicBundle` attribute may appear at most once, and `parseAttributes` rejects duplicates, those iterations cannot change the result. The extra cost is paid on every interop send, as both `sendMessage` and `sendBundle` invoke this function.

Consider breaking out of the loop once the attribute is found, which also makes the returned value independent of the duplicate check performed by `parseAttributes`.

Update: Resolved in [pull request #2347](#).

N-04 Redundant Re-Parsing Of Settlement Proof Per Atomic Interop Leg

The `AtomicInteropProof._authenticateRoot` function processes the same `settlementProof` bytes three times: once via `parseProofMetadata` for the depth check, once inside `proveL2LeafInclusionShared`, and once more via a local `MessageHashing._getProofData` call over identical arguments. The last call repeats both Merkle climbs and the two leaf-hash computations in order to recover `slBlock`, `slChainId`, and `l1BatchTimestamp`, which `proveL2LeafInclusionShared` derives internally but does not return. The `finalProofNode` field can be retrieved by `parseProofMetadata` which already returned it.

This cost is paid per leg, up to `MAX_ATOMIC_FLOW_LEGS` (8).

Consider having `proveL2LeafInclusionShared` return the `ProofData` it computes and consuming it in `_authenticateRoot`, while preserving the existing depth and `finalProofNode` checks.

Update: Resolved in [pull request #2351](#).

N-05 Incorrect Argument Passed To CannotInitiateInteropOnL1

In `InteropCenter.sendBundle`, the `CannotInitiateInteropOnL1` error is raised when an interop transaction is attempted on L1, with `destinationChainId` as its argument.

The checked condition is unrelated to the destination chain that the caller specified. The error declaration in `InteropErrors.sol` uses the same misleading name for the parameter. If this branch is reached, the revert would surface the destination chain id rather than indicating that the call originated on L1, which could be misleading.

Consider dropping the parameter entirely, since `block.chainid` and `L1_CHAIN_ID` are already known (and equal) to anyone debugging the revert.

Update: Resolved in [pull request #2352](#).

N-06 Redundant And Dead Code In Multiple Contracts

Throughout the codebase, several instances of redundant or dead code have been identified:

- The `_getTransferData` function in `AssetRouterBase` can be simplified by removing the `bytes1 encodingVersion` parameter, which is no longer used anywhere in the codebase since `L1AssetRouter._getTransferData` has been removed.
- `L2NativeTokenVault._registerToken` can be removed, as it only calls the base contract's `_registerToken` function.
- In `L2NativeTokenVault.sol`, the comment in [line 184](#) should be removed, as it refers to functions that do not currently exist.
- The `L2NativeTokenVault._ensureAndSaveTokenDeployed` function can be removed, as it contains the exact same code as the function [in the base contract](#).
- In `Mailbox.sol`, the functions regarding proving an L2 log inclusion are not called anywhere within the codebase and can be removed.
- In `L1AssetRouter.sol`, the `onlyNativeTokenVault` modifier is never used and can be removed.
- In `L2AssetRouter.sol`, the `onlyNTV` modifier is not used and can be removed.

Consider removing these redundant parameters, modifiers, functions, and comments to simplify the codebase and improve readability and maintainability.

Update: Resolved in [pull request #2405](#).

N-07 Redundant Read of the Legacy Shared Bridge Address

In `L2GenesisForceDeploymentsHelper._updateAllContracts`, the legacy shared bridge address is obtained through two separate `_getLegacySharedBridge()` calls: one on [line 353](#), passed to `L2AssetRouter.updateL2`, and another on [line 365](#), passed to `L2NativeTokenVault.updateL2`. Both read `L2_LEGACY_SHARED_BRIDGE` from the `L2AssetRouter`.

The second read occurs after `L2AssetRouter.updateL2` has already written to that variable. The two reads currently agree, since `updateL2` writes back verbatim the address it was given, but the code silently relies on this round trip being an identity.

As a good practice, consider reading the address once into a local variable before both calls and passing it to each, so that the two contracts are guaranteed to be configured consistently.

Update: Resolved in [pull request #2395](#).

Conclusion

In this diff audit we reviewed the changes of the v32 release, most notably the introduction of atomic interop, whereby a cross-chain flow spanning several chains either executes on every destination or on none, without a coordinating contract on L1. Alongside it we reviewed the redesign of the chain-local asset bookkeeping and the changes that allow an existing ecosystem to be upgraded in place from v31. In this line of the release the legacy bridging paths are removed entirely, leaving the interop bundle path as the only route for cross-chain transfers, and proof verification on ZKsync OS chains is reduced to a single verifier.

No critical, high, or medium severity issues were identified. The two low severity issues concern edge cases of the atomic interop timeout and refund path rather than the design itself, while the remaining observations are informational and cover documentation accuracy, error reporting, and minor redundancies.

The codebase was of high quality and thoroughly documented, both inline and through a dedicated set of protocol documents describing the atomic interop design, and the new functionality was accompanied by an extensive test suite. The Matter Labs team is appreciated for their responsiveness and for the detailed explanations of design decisions provided throughout the review.

Appendix

Issue Classification

OpenZeppelin classifies smart contract vulnerabilities on a 5-level scale:

- Critical
- High
- Medium
- Low
- Note/Information

Critical Severity

This classification is applied when the issue's impact is catastrophic, threatening extensive damage to the client's reputation and/or causing severe financial loss to the client or users. The likelihood of exploitation can be high, warranting a swift response. Critical issues typically involve significant risks such as the permanent loss or locking of a large volume of users' sensitive assets or the failure of core system functionalities without viable mitigations. These issues demand immediate attention due to their potential to compromise system integrity or user trust significantly.

High Severity

These issues are characterized by the potential to substantially impact the client's reputation and/or result in considerable financial losses. The likelihood of exploitation is significant, warranting a swift response. Such issues might include temporary loss or locking of a significant number of users' sensitive assets or disruptions to critical system functionalities, albeit with potential, yet limited, mitigations available. The emphasis is on the significant but not always catastrophic effects on system operation or asset security, necessitating prompt and effective remediation.

Medium Severity

Issues classified as being of medium severity can lead to a noticeable negative impact on the client's reputation and/or moderate financial losses. Such issues, if left unattended, have a moderate likelihood of being exploited or may cause unwanted side effects in the system.

These issues are typically confined to a smaller subset of users' sensitive assets or might involve deviations from the specified system design that, while not directly financial in nature, compromise system integrity or user experience. The focus here is on issues that pose a real but contained risk, warranting timely attention to prevent escalation.

Low Severity

Low-severity issues are those that have a low impact on the client's operations and/or reputation. These issues may represent minor risks or inefficiencies to the client's specific business model. They are identified as areas for improvement that, while not urgent, could enhance the security and quality of the codebase if addressed.

Notes & Additional Information Severity

This category is reserved for issues that, despite having a minimal impact, are still important to resolve. Addressing these issues contributes to the overall security posture and code quality improvement but does not require immediate action. It reflects a commitment to maintaining high standards and continuous improvement, even in areas that do not pose immediate risks.